

exam_radon

2025-11-24

Init

```
library(emmeans)
```

```
## Welcome to emmeans.  
## Caution: You lose important information if you filter this package's results.  
## See '? untidy'
```

```
library(multcomp)
```

```
## Loading required package: mvtnorm
```

```
## Loading required package: survival
```

```
## Loading required package: TH.data
```

```
## Loading required package: MASS
```

```
##  
## Attaching package: 'TH.data'
```

```
## The following object is masked from 'package:MASS':  
##  
##      geyser
```

```
library(lme4)
```

```
## Loading required package: Matrix
```

```
library(lmerTest)
```

```
##  
## Attaching package: 'lmerTest'
```

```
## The following object is masked from 'package:lme4':  
##  
##      lmer
```

```
## The following object is masked from 'package:stats':  
##  
##     step
```

```
library(ggplot2)  
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following object is masked from 'package:MASS':  
##  
##     select
```

```
## The following objects are masked from 'package:stats':  
##  
##     filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##     intersect, setdiff, setequal, union
```

```
library(predictmeans)
```

```
## Loading required package: glmmTMB
```

```
## Loading required package: nlme
```

```
##  
## Attaching package: 'nlme'
```

```
## The following object is masked from 'package:dplyr':  
##  
##     collapse
```

```
## The following object is masked from 'package:lme4':  
##  
##     lmList
```

```
library(lattice) # to visualize random effects
```

```
# setwd("C:/Users/huste/Documents/Github/businessAnalytics/Correlated Data/Exam")  
setwd("/home/husted42/Documents/github/businessAnalytics/Correlated Data/Exam/")  
  
getwd()
```

```
## [1] "/home/husted42/Documents/github/businessAnalytics/Correlated Data/Exam"
```

```
list.files()
```

```
## [1] "radon_code_files" "radon_code.html" "radon_code.Rmd" "Radon_data.R"
## [5] "Radon_MN.csv"      "rat_code.html"    "rat_code.Rmd"   "Rats_Box.txt"
```

```
df<-read.table("Radon_MN.csv",header = T)
head(df)
```

```
##   idnum state state2 stfips   zip region typebldg floor room basement windoor
## 1  5081    MN     MN     27 55735      5        1     1     3          N      NA
## 2  5082    MN     MN     27 55748      5        1     0     4          Y      NA
## 3  5083    MN     MN     27 55748      5        1     0     4          Y      NA
## 4  5084    MN     MN     27 56469      5        1     0     4          Y      NA
## 5  5085    MN     MN     27 55011      3        1     0     4          Y      NA
## 6  5086    MN     MN     27 55014      3        1     0     4          Y      NA
##   rep stratum wave starttm stoptm startdt stopdt activity pcterr   adjwt
## 1   2       4   41     930    930   12088   12288      2.2    9.7 1146.4992
## 2   5       2   40    1615   1615   11888   12088      2.2   14.5  471.3662
## 3   3       2   42    1030   1515   20288   21188      2.9    9.6  433.3167
## 4   2       2   24    1410   1410  122987  123187      1.0   24.3  461.6237
## 5   3       2   40     600    600   12888   13088      3.1   13.8  433.3167
## 6   5       2   22     900   1300  120287  121187      2.5   12.8  471.3662
##   dupflag zipflag cntyfips county x      county.name      y      u.full
## 1      1       0       1     1 1 AITKIN      0.7884574 -0.6890476
## 2      0       0       1     1 0 AITKIN      0.7884574 -0.6890476
## 3      0       0       1     1 0 AITKIN      1.0647107 -0.6890476
## 4      0       0       1     1 0 AITKIN      0.0000000 -0.6890476
## 5      0       0       3     2 0 ANOKA      1.1314021 -0.8473129
## 6      0       0       3     2 0 ANOKA      0.9162907 -0.8473129
```

I. Exploratory analysis

```
df$county <- as.factor(df$county)
df$floor <- as.factor(df$floor)
unique(df$county)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
## [51] 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75
## [76] 76 77 78 79 80 81 82 83 84 85
## 85 Levels: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 ... 85
```

```
unique(df$floor)
```

```
## [1] 1 0
## Levels: 0 1
```

```
summary(df$u)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.88183 -0.47467 -0.09652 -0.13171  0.18324  0.52802
```

```
summary(df$y)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -2.3026  0.6419  1.2809  1.2246  1.7918  3.8754
```

We see that there are 85 different counties and floor is a binary variable 0 meaning basement and 1 meaning not basement.

```
xtabs(~ county, df)
```

```
## county
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
##  4 52  3  7  4  3 14  4 10  6  5  4  6 14  4  2  4 12 63  3
## 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
##  9  6  2  9 14 105  6  5  3 11  5  4  4  3  7  2  9  4  5  4
## 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
##  8  1  9  7 13  5  2  9 13  1  4  3  3 23  8  3  6  4  4  2
## 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
## 32  5  3 11  2 14 13  8  4 116 25 10  2  4  3  4  7  5  4 46
## 81 82 83 84 85
##  3  1 13 13  2
```

The data is not balanced

```
xtabs(~ county, df)
```

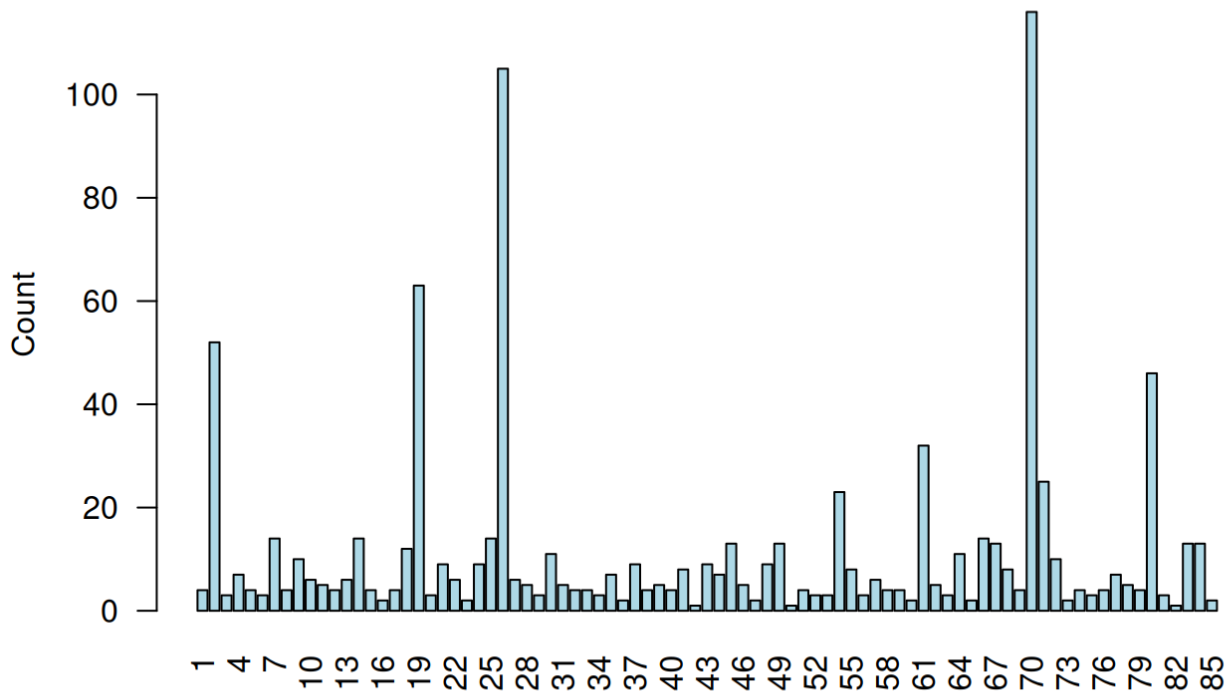
```
## county
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
##  4 52  3  7  4  3 14  4 10  6  5  4  6 14  4  2  4 12 63  3
## 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
##  9  6  2  9 14 105  6  5  3 11  5  4  4  3  7  2  9  4  5  4
## 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
##  8  1  9  7 13  5  2  9 13  1  4  3  3 23  8  3  6  4  4  2
## 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
## 32  5  3 11  2 14 13  8  4 116 25 10  2  4  3  4  7  5  4 46
## 81 82 83 84 85
##  3  1 13 13  2
```

```
xtabs(~ floor, df)
```

```
## floor
##  0  1
## 766 153
```

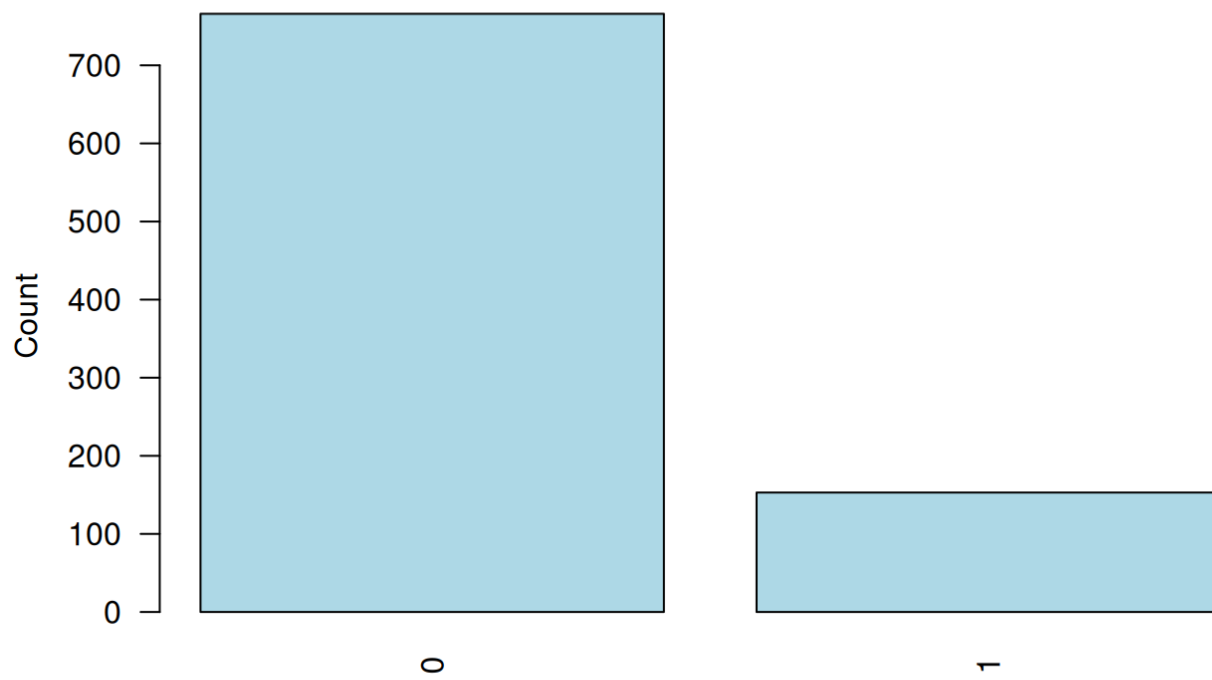
```
barplot(  
  xtabs(~ county, df),  
  las = 2,                # rotate county labels for readability  
  col = "lightblue",  
  main = "Number of Observations per County",  
  ylab = "Count"  
)
```

Number of Observations per County



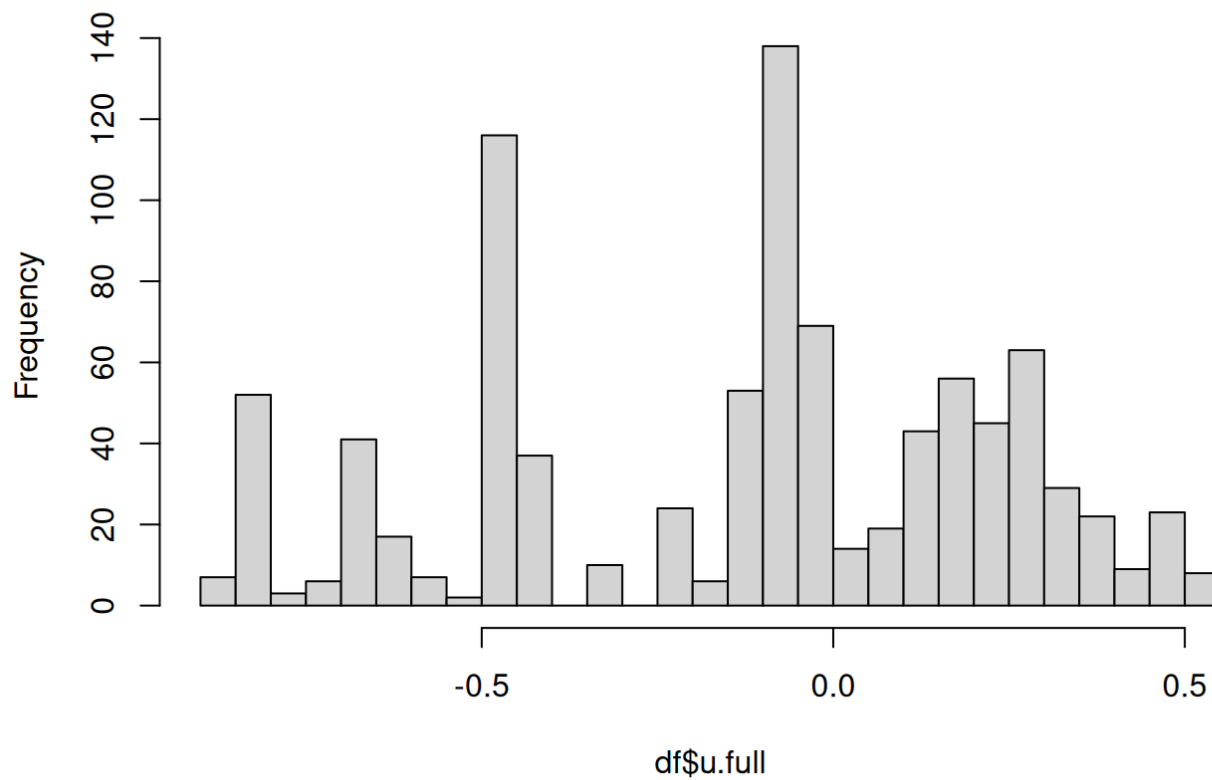
```
barplot(  
  xtabs(~ floor, df),  
  las = 2,                # rotate county labels for readability  
  col = "lightblue",  
  main = "Number of Observations per floor",  
  ylab = "Count"  
)
```

Number of Observations per floor



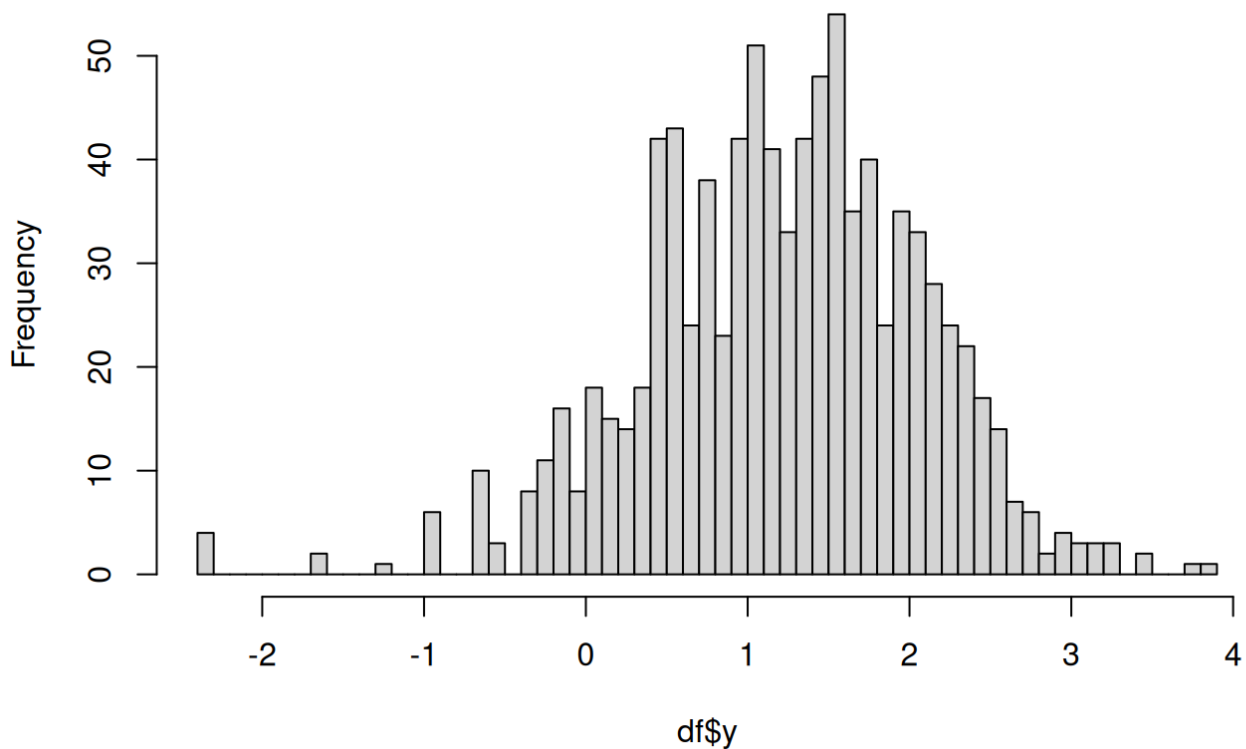
```
hist(df$u.full, breaks = 50, col = "lightgray", main = "Distribution of u.full")
```

Distribution of u.full



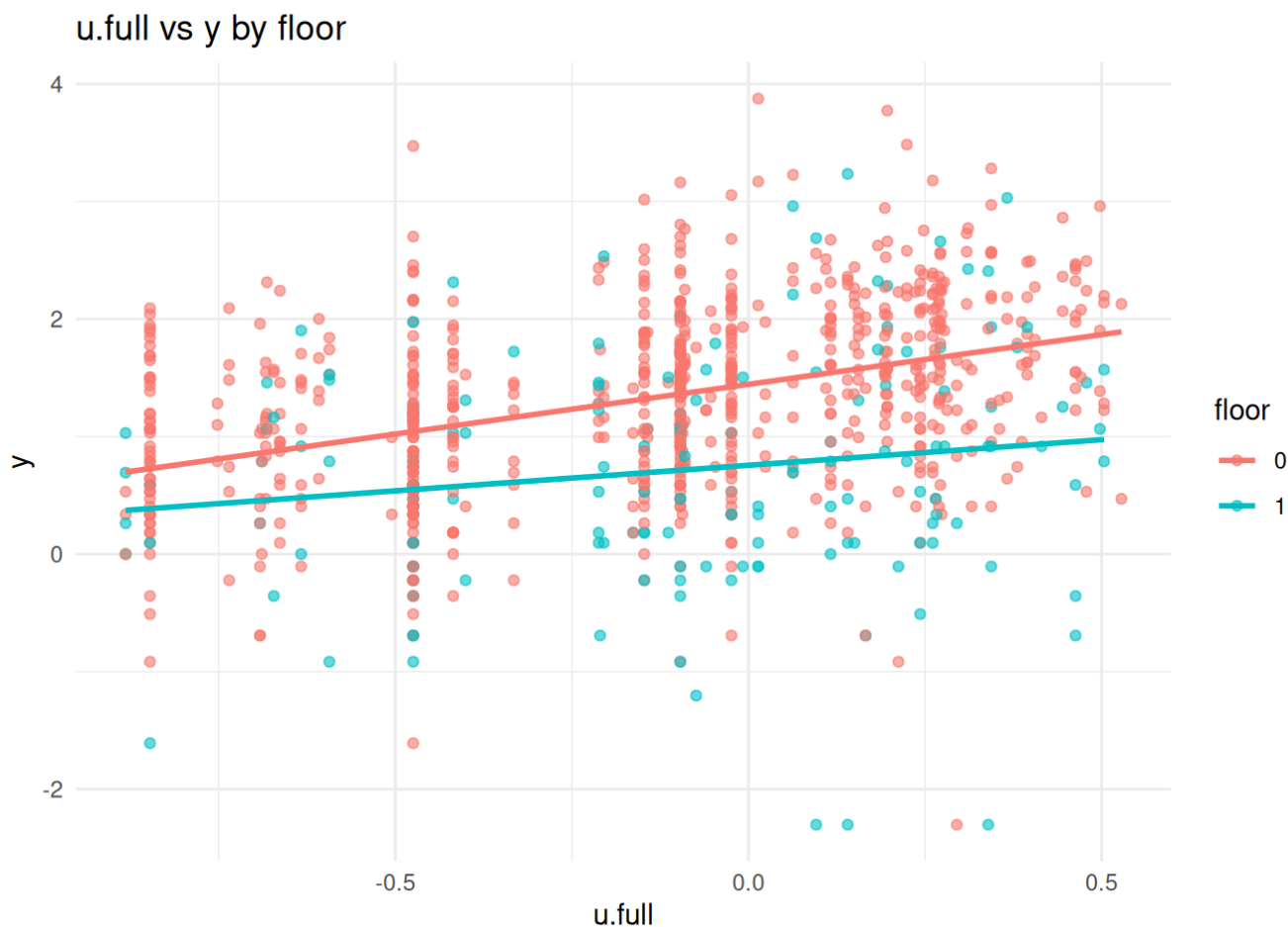
```
hist(df$y, breaks = 50, col = "lightgray", main = "Distribution of log(radon)")
```

Distribution of log(radon)



```
ggplot(df, aes(x = u.full, y = y, color = floor)) +  
  geom_point(alpha = 0.6) +  
  geom_smooth(method = "lm", se = FALSE) +  
  theme_minimal() +  
  labs(title = "u.full vs y by floor")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
cor(df[, c("u.full", "y")])
```

```
##          u.full          y
## u.full 1.0000000 0.3219591
## y      0.3219591 1.0000000
```

We see that neither of the categorical items are balanced. With 5 counties having many samples: 26 and 70 are above 100 while 2, 19 and 80 are above 50. And 83% of the data having floor = 0.

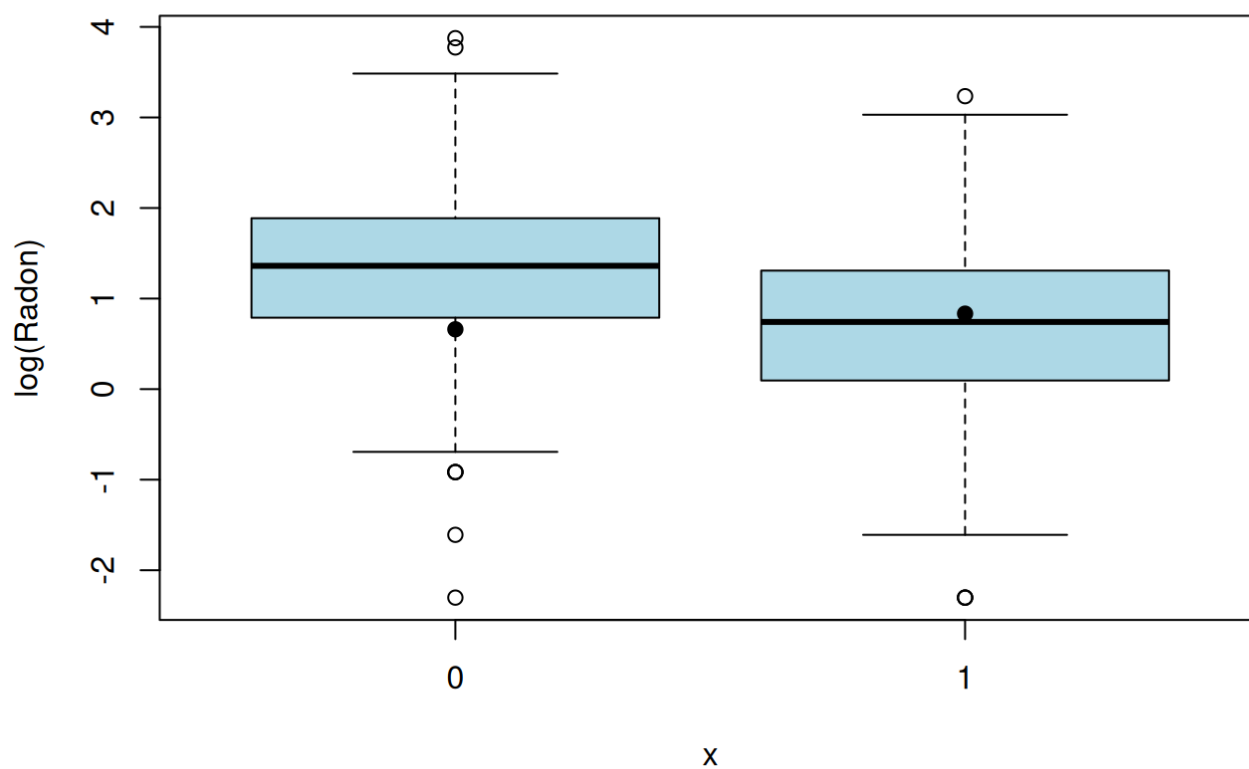
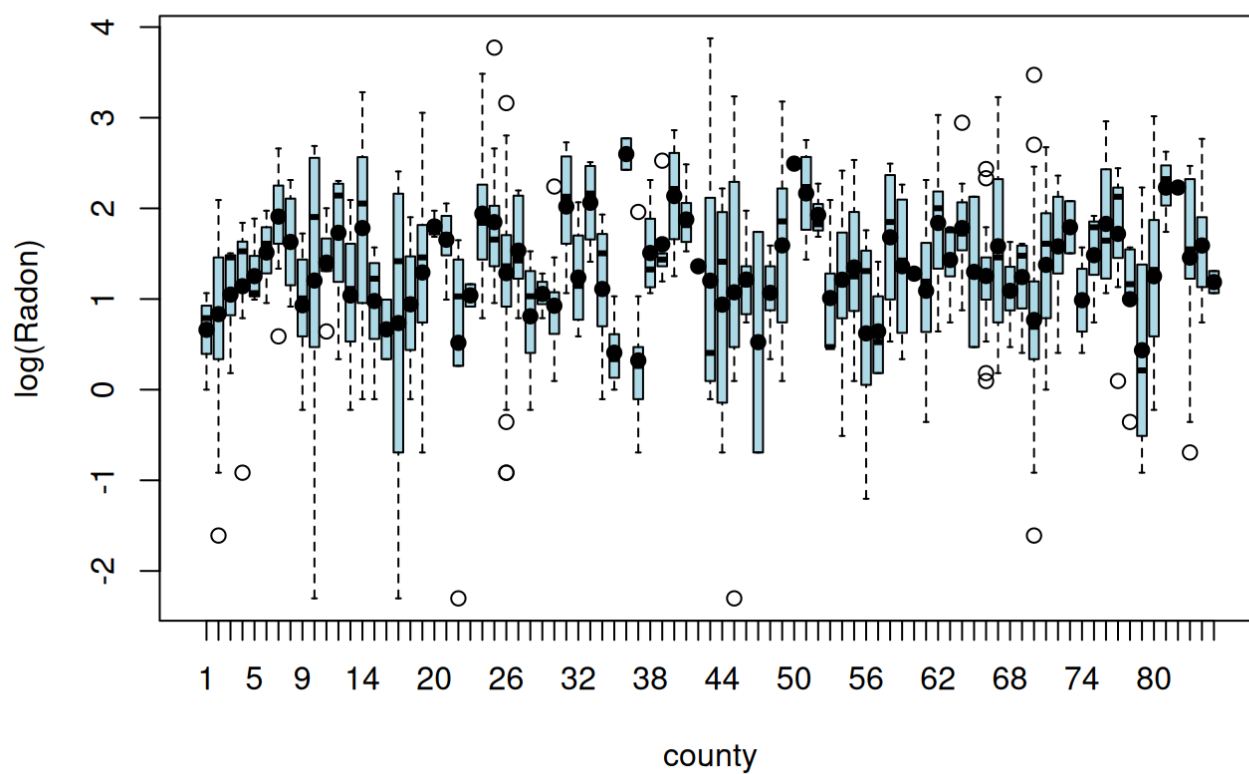
```
vars <- c("x", "county")

# par(mfrow = c(2, 2))

for (v in vars) {
  formula <- as.formula(paste("y ~", v))

  boxplot(formula, data = df,
    main = paste("log(Radon) by", v),
    xlab = v,
    ylab = "log(Radon)",
    col = "lightblue")

  means <- tapply(df$y, df$county, mean)
  points(1:length(means), means, pch = 19, col = "black")
}
```

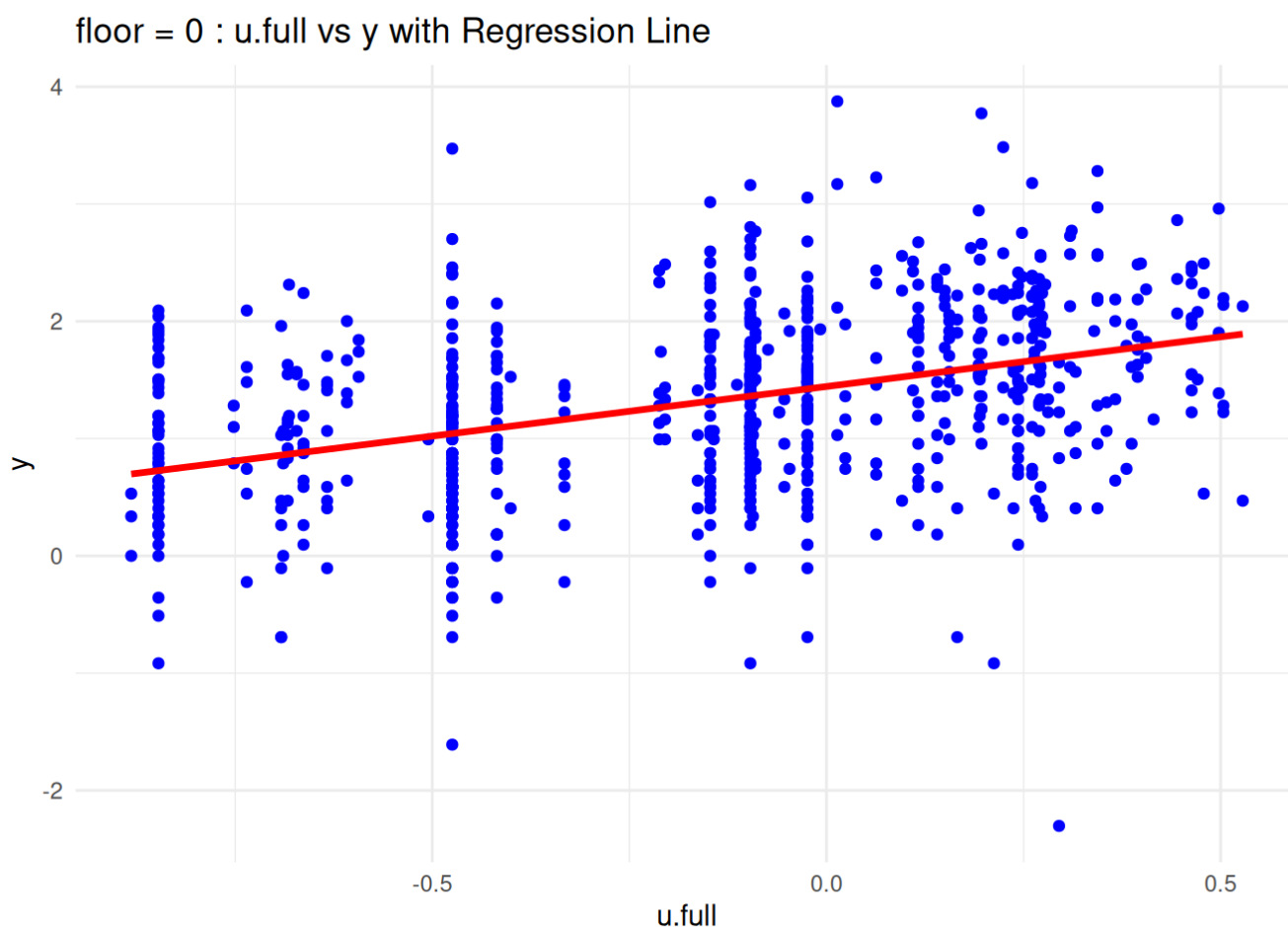

log(Radon) by x**log(Radon) by county**

```
par(mfrow = c(1, 1))
```

It's not clear whether there is a significant between group variance, we see that we have overlap between the inter quartile ranges for the two groups.

```
radon_floor_0 <- df %>%  
  filter(floor == 0)  
  
ggplot(radon_floor_0, aes(x = u.full, y = y)) +  
  geom_point(color = "blue") +  
  geom_smooth(method = "lm", se = FALSE, color = "red", linewidth = 1.2) +  
  labs(  
    x = "u.full",  
    y = "y",  
    title = "floor = 0 : u.full vs y with Regression Line"  
  ) +  
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



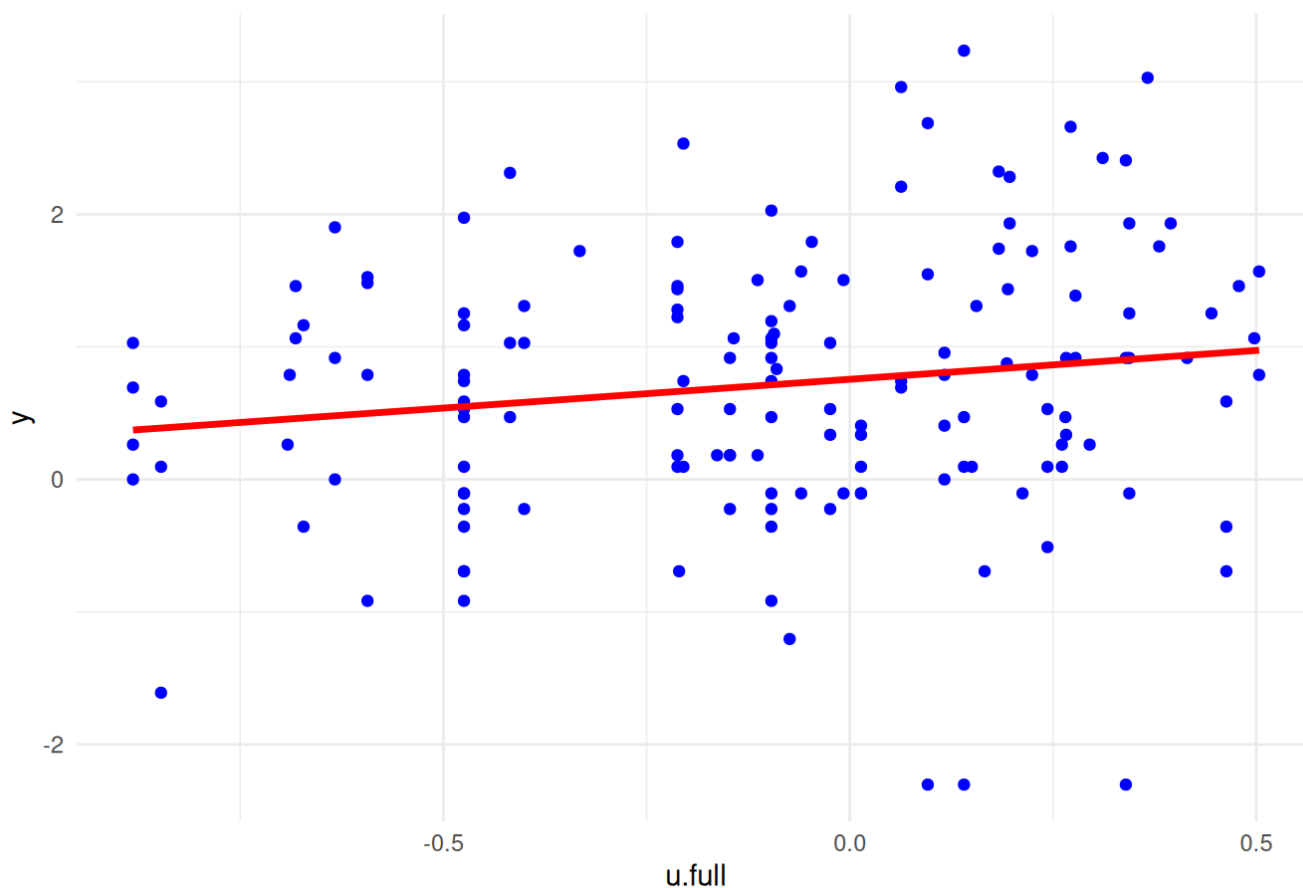
```
model <- lm(y ~ u.full, data = radon_floor_0)  
summary(model)
```

```
##  
## Call:  
## lm(formula = y ~ u.full, data = radon_floor_0)  
##  
## Residuals:  
##      Min       1Q   Median       3Q      Max   
## -3.9963 -0.4546  0.0371  0.4725  2.4296   
##  
## Coefficients:  
##              Estimate Std. Error t value Pr(>|t|)      
## (Intercept)  1.44407    0.02779   51.97  <2e-16 ***   
## u.full       0.84628    0.07099   11.92  <2e-16 ***   
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## Residual standard error: 0.7192 on 764 degrees of freedom  
## Multiple R-squared:  0.1568, Adjusted R-squared:  0.1557   
## F-statistic: 142.1 on 1 and 764 DF,  p-value: < 2.2e-16
```

```
radon_floor_0 <- df %>%  
  filter(floor == 1)  
  
ggplot(radon_floor_0, aes(x = u.full, y = y)) +  
  geom_point(color = "blue") +  
  geom_smooth(method = "lm", se = FALSE, color = "red", linewidth = 1.2) +  
  labs(  
    x = "u.full",  
    y = "y",  
    title = "floor = 1 : u.full vs y with Regression Line"  
  ) +  
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

floor = 1 : u.full vs y with Regression Line



```
model <- lm(y ~ u.full, data = radon_floor_0)
summary(model)
```

```
##
## Call:
## lm(formula = y ~ u.full, data = radon_floor_0)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.2056 -0.6213  0.0132  0.6176  2.4182
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.75550    0.08294   9.109 4.71e-16 ***
## u.full         0.43457    0.22326   1.946  0.0535 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9903 on 151 degrees of freedom
## Multiple R-squared:  0.02448,    Adjusted R-squared:  0.01802
## F-statistic: 3.789 on 1 and 151 DF,  p-value: 0.05345
```

II. Modeling

Building the model

Target : y Uranium : u.full Floor : x County : county ### Model 0 - Mean We start with a simple model and build up from there. We see that the mean is significantly different from 0. We do not accept $H_0 : = 0$

$$y_i = \mu + \epsilon_i, \epsilon_i \sim N(0, \sigma^2)$$

```
model_0 <- lm(y ~ 1, df)
anova(model_0)
```

```
## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value Pr(>F)
## Residuals 918 668.46  0.72817
```

```
summary(model_0)
```

```
##
## Call:
## lm(formula = y ~ 1, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5272 -0.5828  0.0563  0.5671  2.6507
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.22462     0.02815   43.51  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8533 on 918 degrees of freedom
```

Model 1 - Floor

$$y_i = \mu + \alpha_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

```
model_1 <- lm(y ~ floor, df)
anova(model_1)
```

```
## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## floor       1  47.98   47.983   70.914 < 2.2e-16 ***
## Residuals 917 620.47    0.677
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Model 2 - Uranium

$$y_i = \mu + \alpha_i + \beta x_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

```
model_2 <- lm(y ~ floor + u.full , df)
anova(model_2)
```

```
## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## floor      1  47.98   47.983   80.489 < 2.2e-16 ***
## u.full     1  74.41   74.412  124.824 < 2.2e-16 ***
## Residuals 916 546.06    0.596
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Model 3 - Random effect

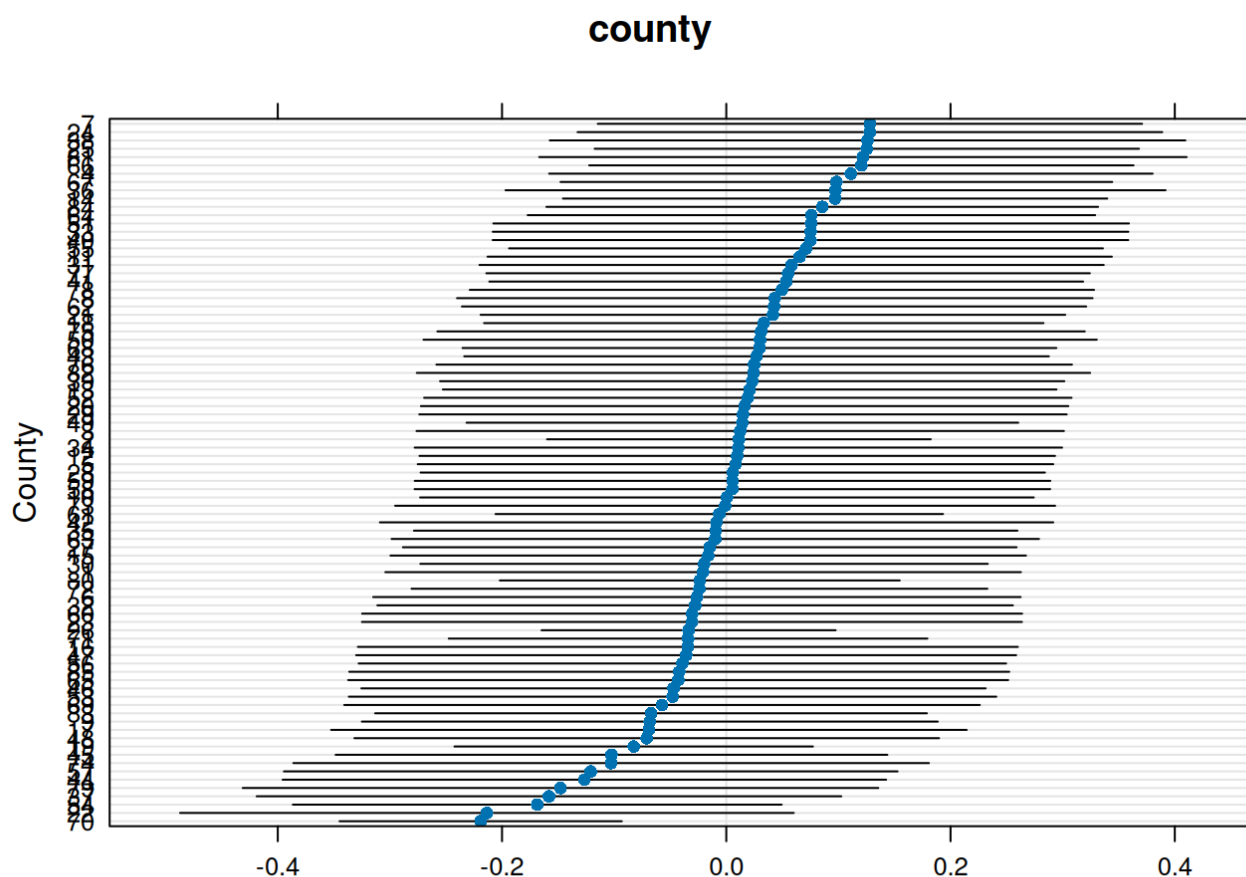
$$y_i = \mu + \alpha_i + \beta x_i + \gamma_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma_\epsilon^2), \quad \gamma_i \sim N(0, \sigma_\gamma^2)$$

```
model_3 <- lmer(y ~ floor + u.full + (1|county) , df)
ranova(model_3)
```

```
## ANOVA-like table for random-effects: Single term deletions
##
## Model:
## y ~ floor + u.full + (1 | county)
##           npar logLik    AIC    LRT Df Pr(>Chisq)
## <none>         5 -1067.1 2144.2
## (1 | county)    4 -1071.1 2150.1  7.9559  1  0.004793 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
dotplot(ranef(model_3, condVar=TRUE), strip = FALSE, ylab="County")
```

```
## $county
```



```
ranef_model_3 <- ranef(model_3, condVar=TRUE)
```

Model 4 & 5 - Random slopes

We add a random slope to the model to allow the impact of the fixed effects to vary across county.

Random slope for predictor floor By including a random slope for the predictor x , we allow its effect on radon levels to depend on the county. This tests whether counties differ in how strongly basement measurements raise or lower radon levels.

```
model_4 <- lmer(y ~ x + u.full + (x|county) , df)
ranova(model_4)
```

```
## ANOVA-like table for random-effects: Single term deletions
##
## Model:
## y ~ x + u.full + (x | county)
##               npar logLik   AIC   LRT Df Pr(>Chisq)
## <none>           7 -1064.3 2142.6
## x in (x | county)  5 -1067.1 2144.2 5.5459  2    0.06248 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

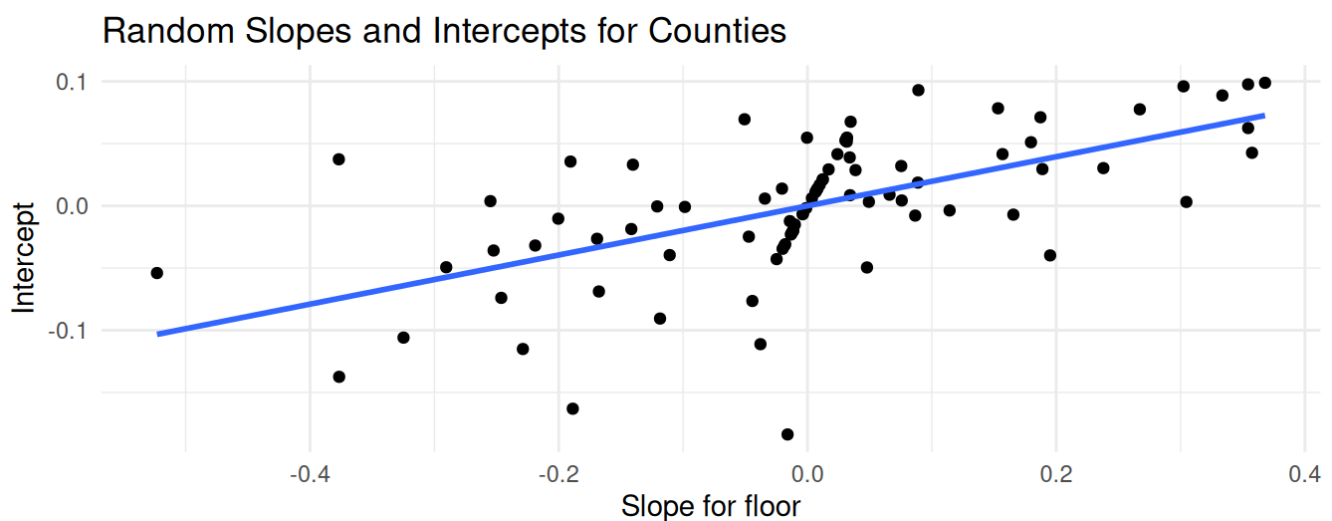
The addition of a random-slope term is not statistically significant ($p = 0.062$). Therefore, the basement effect appears to be consistent across counties, and the random-intercept model is adequate.

```

ranef_m4 <- ranef(model_4)$county
library(ggplot2)
ggplot(ranef_m4, aes(x = x, y = `(Intercept)`)) +
  geom_point() +
  geom_smooth(method = "lm",
              se = FALSE) +
  labs(title = "Random Slopes and Intercepts for Counties",
       x = "Slope for floor",
       y = "Intercept") +
  # xlim(-0.6, 0.6) +
  # ylim(-0.4, 0.4) +
  coord_fixed() + # <-- makes width = height
  theme_minimal()

```

```
## `geom_smooth()` using formula = 'y ~ x'
```



The above graph show the the slopes and intercept for each cluster. We don't see clear patterns in the points exeot one straight line. I won't look into further detail as the effect of the random slope are not significant.

Random slope for predictor u.full We skip this since there is no within-county variation.

Full model

$$y_i = \mu + \alpha_i + \beta x_i + \gamma_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma_\epsilon^2), \quad \gamma_i \sim N(0, \sigma_\gamma^2)$$


```
model <- lmer(y ~ floor + u.full + (1|county) , df)
summary(model)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ floor + u.full + (1 | county)
## Data: df
##
## REML criterion at convergence: 2134.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -4.9673 -0.6117  0.0274  0.6555  3.3848
##
## Random effects:
## Groups   Name      Variance Std.Dev.
## county   (Intercept) 0.02446  0.1564
## Residual                0.57523  0.7584
## Number of obs: 919, groups: county, 85
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)   1.46576    0.03794  55.34577  38.633 < 2e-16 ***
## floor1        -0.66824    0.06880  902.87114  -9.713 < 2e-16 ***
## u.full         0.72027    0.09176  45.99114   7.849 4.94e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) floor1
## floor1 -0.357
## u.full  0.145 -0.009
```

$$\hat{\sigma}_{\epsilon} = 0.7584, \quad \hat{\sigma}_{\gamma} = 0.1564$$

We see that only 4 % of the variance is explained by the random effect.

$$\frac{0.024}{0.575 + 0.024} = 4\%$$

```
confint(model)
```

```
## Computing profile confidence intervals ...
```

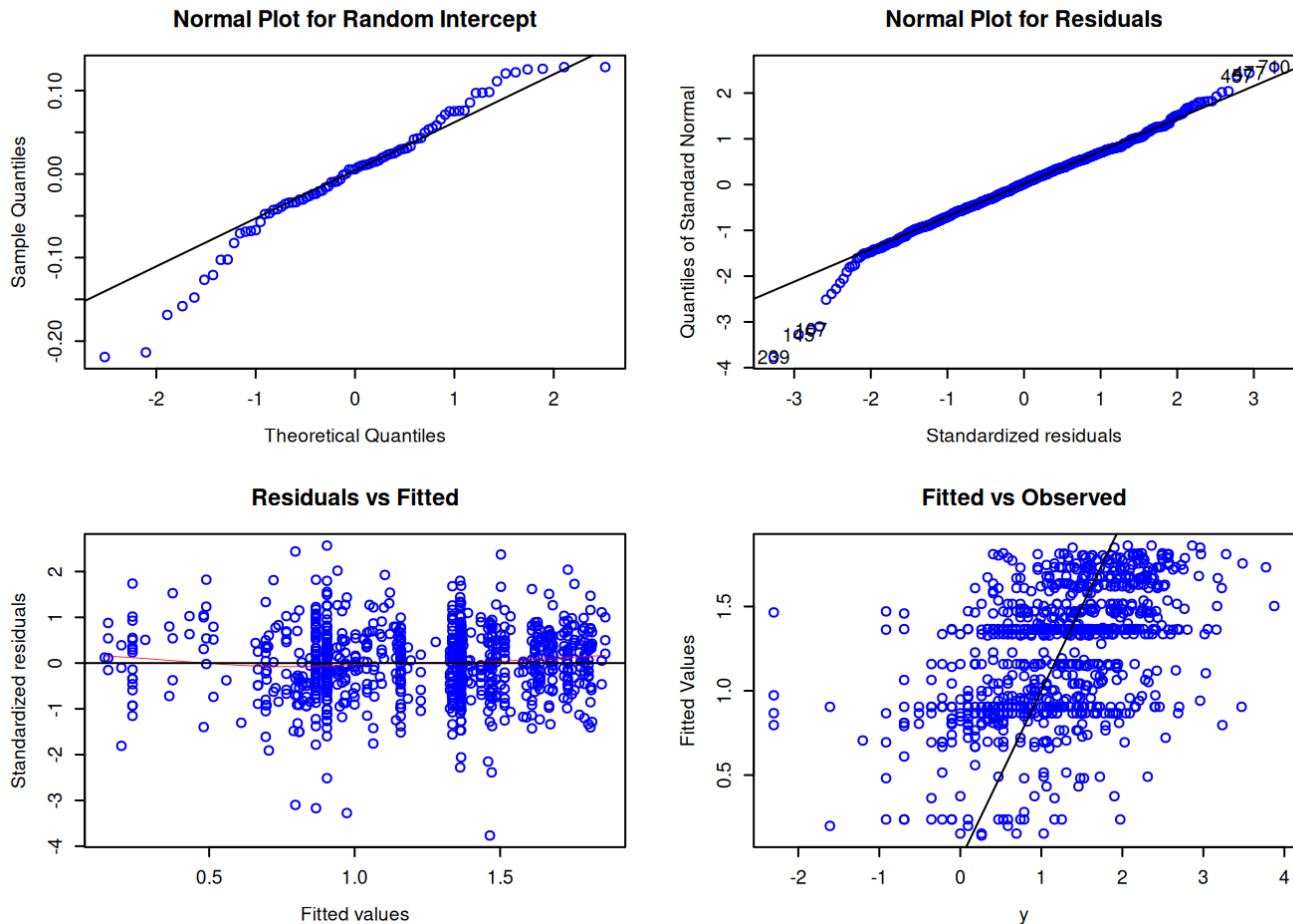
```
##              2.5 %      97.5 %
## .sig01        0.05277016  0.2454198
## .sigma        0.72351612  0.7957389
## (Intercept)   1.39137411  1.5415136
## floor1       -0.80224373 -0.5307890
## u.full        0.53732370  0.8992722
```

Given our observation we are 95% confident that the population mean $\hat{\mu}$ are between (1.39, 1.54) given

floor = 0 and u.full = 0. The slope \$ \$ are between (0.54, 0.90). And that the effect of floor=1 are between (-08, -0.53)

Diagnostics

```
residplot(model)
```



```
df[299, ]
```

```
##      idnum state state2 stfips  zip region typebldg floor room basement windowor
## 299  5379   MN     MN    27 55413     3      1     0    4         Y      NA
##      rep stratum wave starttm stoptm startdt stopdt activity pcterr   adjwt
## 299   2      5   37   2000   2000   11088  11288     1.9   9.8 2246.969
##      dupflag zipflag cntyfips county x          county.name          y
## 299      0      0      53    26 0 HENNEPIN          0.6418539
##      u.full
## 299 -0.09652081
```

```
df[149, ]
```

```
##      idnum state state2 stfips   zip region typebldg floor room basement window
## 149  5229    MN     MN     27 56401      2        1     1     2              NA
##      rep stratum wave starttm stoptm startdt stopdt activity pterr   adjwt
## 149   4        3   44     900   900   12388  12588         1   18.7 990.4116
##      dupflag zipflag cntyfips county x          county.name y      u.full
## 149        1        0       35    18 1 CROW WING           0 -0.6333907
```

```
predict(model, newdata = df[299, ])
```

```
##      299
## 1.36263
```

```
predict(model, newdata = df[149, ])
```

```
##      149
## 0.3747311
```

```
predict(model, newdata = df[299, ], re.form = NA)
```

```
##      299
## 1.396242
```

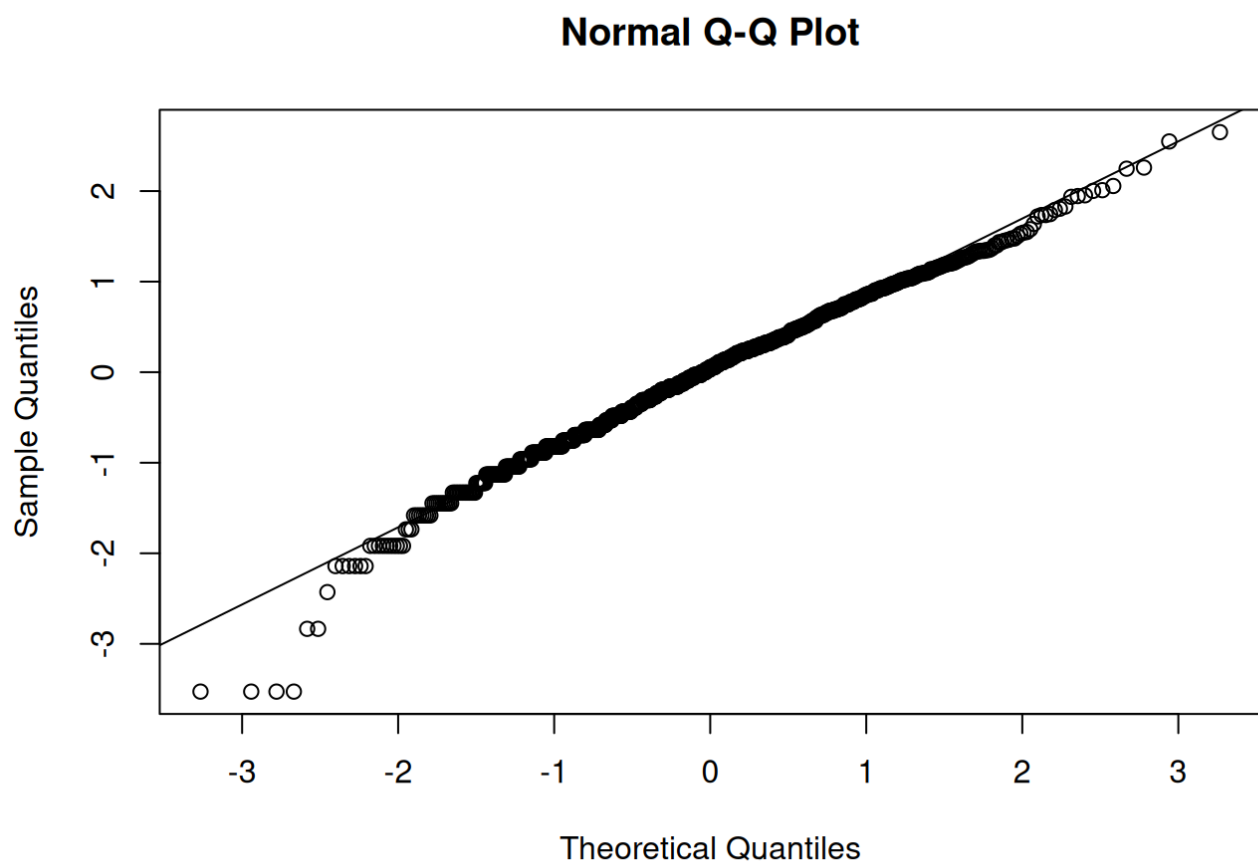
```
predict(model, newdata = df[149, ], re.form = NA)
```

```
##      149
## 0.3413072
```

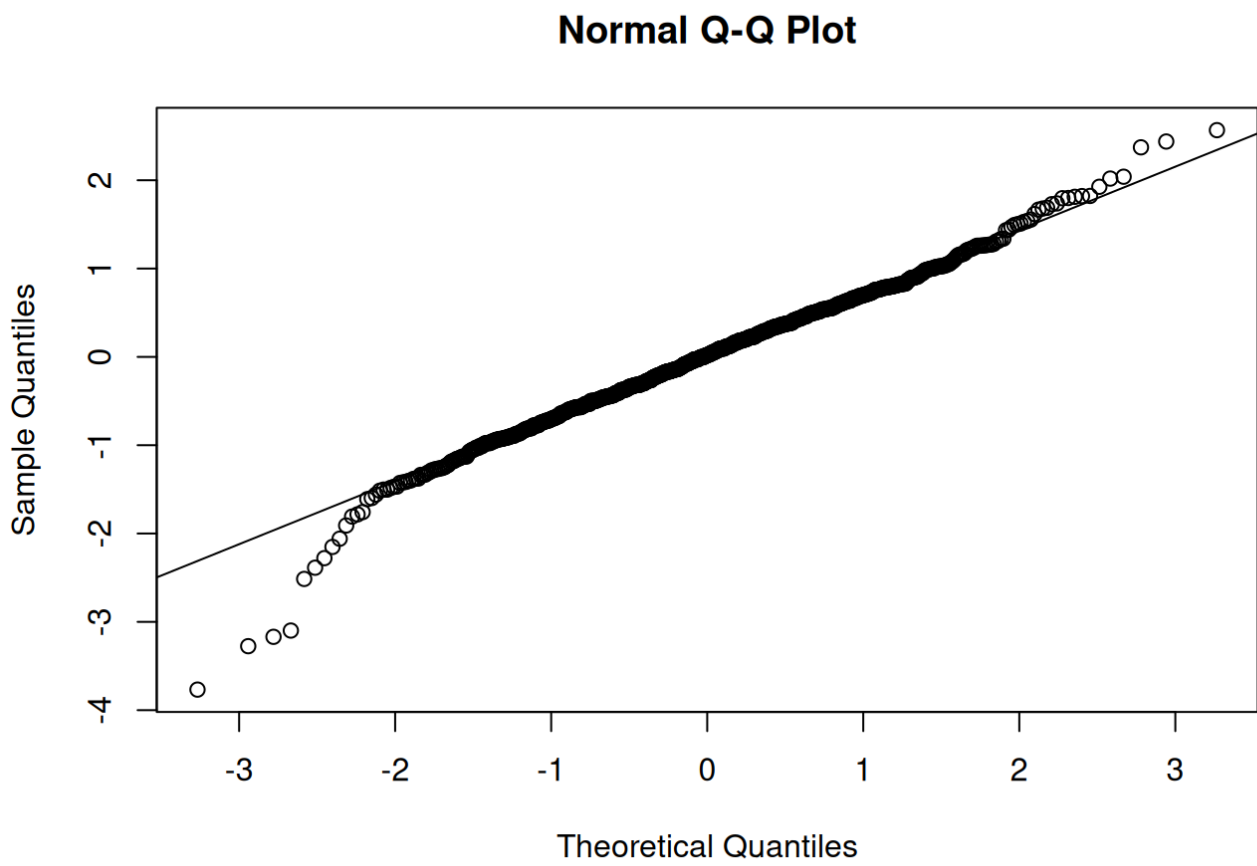
Comparison

Look at the change to the diagnostic plots as complexity increases

```
qqnorm(resid(model_0))
qqline(resid(model_0))
```



```
qqnorm(resid(model_3))  
qqline(resid(model_3))
```



Comparison table

```
AIC(model_0, model_1, model_2, model_3)
```

```
##           df      AIC
## model_0    2 2319.479
## model_1    3 2253.025
## model_2    4 2137.621
## model_3    5 2144.186
```

```
BIC(model_0, model_1, model_2, model_3)
```

```
##           df      BIC
## model_0    2 2329.126
## model_1    3 2267.495
## model_2    4 2156.914
## model_3    5 2168.303
```

```
logLik(model_0)
```

```
## 'log Lik.' -1157.74 (df=2)
```

```
logLik(model_1)
```

```
## 'log Lik.' -1123.512 (df=3)
```

```
logLik(model_2)
```

```
## 'log Lik.' -1064.81 (df=4)
```

```
logLik(model_3)
```

```
## 'log Lik.' -1067.093 (df=5)
```

Model	df	logLik	AIC	BIC
model_0	2	-1157.740	2319.479	2329.126
model_1	3	-1123.512	2253.025	2267.495
model_2	4	-1064.810	2137.621	2156.914
model_3	5	-1067.093	2144.186	2168.303

We want AIC and BIC to be as small as possible, but it seems that both AIC and BIC is larger for model 2 than 3.

$$\text{AIC} = -2 \log L(\hat{\theta}) + 2p,$$

$$\text{BIC} = -2 \log L(\hat{\theta}) + (\log n) p$$

This is partially due to the increase in the complexity penalty since p increases by 1, but we also see an increase in log likelihood.

Even though the variance of the residuals decrease from $0.772^2 = 0.596$ in model_2 to $0.758^2 = 0.575$ in model_3, we get an additional variance from the random effect of $0.156^2 = 0.024$ meaning that the overall variance increases high decreases the log-likelihood function.

```
summary(model_2)
```

```
##
## Call:
## lm(formula = y ~ floor + u.full, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.9676 -0.4809  0.0267  0.4993  2.4297
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.43489    0.02953  48.593  <2e-16 ***
## floor1      -0.64589    0.06843  -9.438  <2e-16 ***
## u.full       0.78004    0.06982  11.172  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7721 on 916 degrees of freedom
## Multiple R-squared:  0.1831, Adjusted R-squared:  0.1813
## F-statistic: 102.7 on 2 and 916 DF,  p-value: < 2.2e-16
```

```
summary(model_3)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ floor + u.full + (1 | county)
##      Data: df
##
## REML criterion at convergence: 2134.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -4.9673 -0.6117  0.0274  0.6555  3.3848
##
## Random effects:
##      Groups      Name      Variance Std.Dev.
## county (Intercept) 0.02446  0.1564
## Residual          0.57523  0.7584
## Number of obs: 919, groups: county, 85
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)  1.46576    0.03794  55.34577  38.633  < 2e-16 ***
## floor1      -0.66824    0.06880  902.87114  -9.713  < 2e-16 ***
## u.full       0.72027    0.09176  45.99114   7.849 4.94e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) floor1
## floor1 -0.357
## u.full  0.145 -0.009
```

III. Results

III a) Present a plot analogous to the one in Figure 1, with the estimated response means per county


```

model <- lmer(y ~ floor + u.full + (1|county) , df)
df_pred <- df
df_pred_before_random <- df
df_pred$y <- predict(model, newdata = df)
df_pred_before_random$y <- predict(model_2, newdata = df)

invisible({
  attach(df)
  (J <- length(unique(county.name)))
  (ybarbar = mean(y))

  (sample.size <- as.vector(table(county)))
  (sample.size.jittered <- sample.size*exp(runif (J, -.1, .1)))
  (cty.mns = tapply(y,county,mean))
  (cty.vars = tapply(y,county,var))
  (cty.sds = mean(sqrt(cty.vars[!is.na(cty.vars)]))/sqrt(sample.size))
  (cty.sds.sep = sqrt(tapply(y,county,var)/sample.size))

  par(mfrow=c(1,1))
  plot (sample.size.jittered, cty.mns, cex.lab=.9, cex.axis=1,
        xlab="sample size in county j",
        ylab="Mean log radon in county j",
        pch=20, log="x", cex=.3, mgp=c(1.5,.5,0),
        ylim=c(0,3.2), yaxt="n", xaxt="n")
  axis (1, c(1,3,10,30,100), cex.axis=.9, mgp=c(1.5,.5,0))
  axis (2, seq(0,3), cex.axis=.9, mgp=c(1.5,.5,0))
  for (j in 1:J){
    lines (rep(sample.size.jittered[j],2),
           cty.mns[j] + c(-1,1)*cty.sds[j], lwd=.5)
    #      cty.mns[j] + c(-1,1)*mean(cty.sds[!is.na(cty.sds)]), lwd=.5)
  }
  abline(h=mean(cty.mns),lwd=.5 )
  title("Observed mean response values per county (data)",cex.main=.9, line=1)

  cty.mns[cty.mns==max(cty.mns)]
  cty.mns[cty.mns==min(cty.mns)]

  points(sample.size.jittered[36],cty.mns[36],cex=2) #max
  points(sample.size.jittered[37],cty.mns[37],cex=2) #min
  detach(df)

  attach(df_pred)
  (J <- length(unique(county.name)))
  (ybarbar = mean(y))

  (sample.size <- as.vector(table(county)))
  (sample.size.jittered <- sample.size*exp(runif (J, -.1, .1)))
  (cty.mns = tapply(y,county,mean))
  (cty.vars = tapply(y,county,var))
  (cty.sds = mean(sqrt(cty.vars[!is.na(cty.vars)]))/sqrt(sample.size))
  (cty.sds.sep = sqrt(tapply(y,county,var)/sample.size))

```

```

par(mfrow=c(1,1))
plot (sample.size.jittered, cty.mns, cex.lab=.9, cex.axis=1,
      xlab="sample size in county j",
      ylab="Mean log radon in county j",
      pch=20, log="x", cex=.3, mgp=c(1.5,.5,0),
      ylim=c(0,3.2), yaxt="n", xaxt="n")
axis (1, c(1,3,10,30,100), cex.axis=.9, mgp=c(1.5,.5,0))
axis (2, seq(0,3), cex.axis=.9, mgp=c(1.5,.5,0))
for (j in 1:J){
  lines (rep(sample.size.jittered[j],2),
        cty.mns[j] + c(-1,1)*cty.sds[j], lwd=.5)
  #      cty.mns[j] + c(-1,1)*mean(cty.sds[!is.na(cty.sds)]), lwd=.5)
}
abline(h=mean(cty.mns),lwd=.5 )
title("Observed mean response values per county full_model",cex.main=.9, line=1)

cty.mns[cty.mns==max(cty.mns)]
cty.mns[cty.mns==min(cty.mns)]

points(sample.size.jittered[36],cty.mns[36],cex=2) #max
points(sample.size.jittered[37],cty.mns[37],cex=2) #min

# print(cty.sds)

detach(df_pred)

attach(df_pred_before_random)
(J <- length(unique(county.name)))
(ybarbar = mean(y))

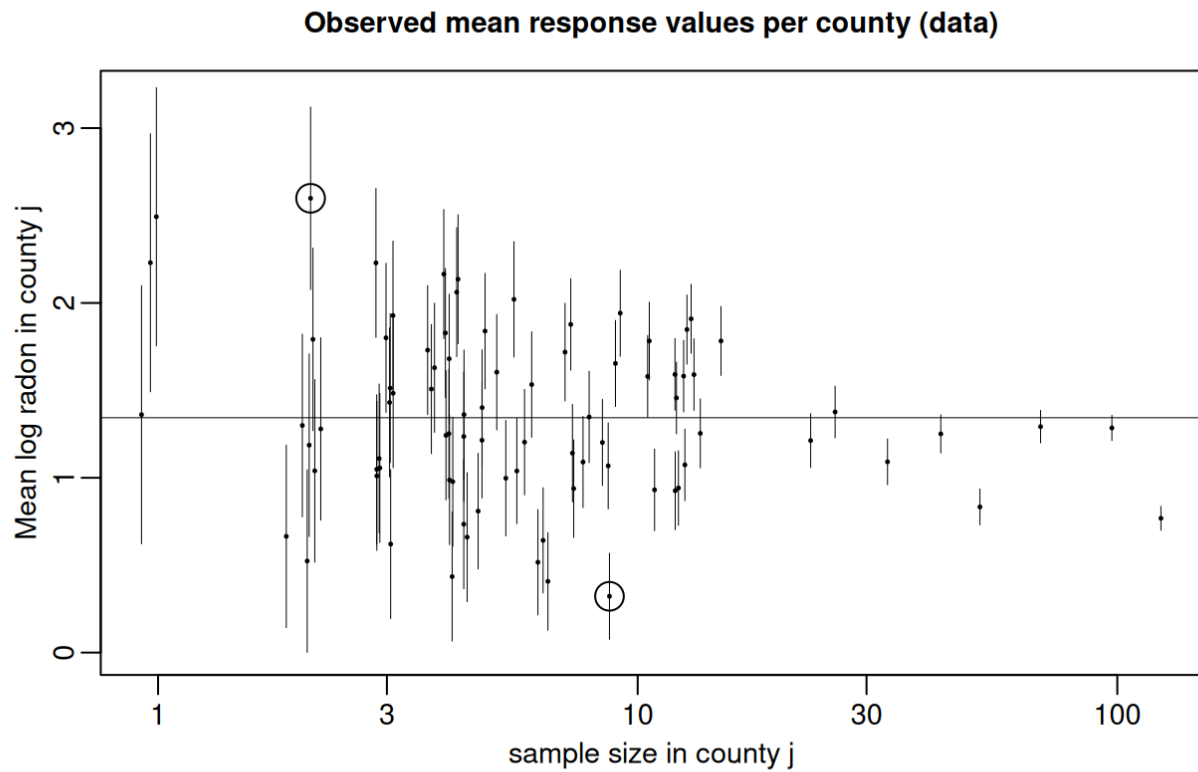
(sample.size <- as.vector(table(county)))
(sample.size.jittered <- sample.size*exp(runif (J, -.1, .1)))
(cty.mns = tapply(y,county,mean))
(cty.vars = tapply(y,county,var))
(cty.sds = mean(sqrt(cty.vars[!is.na(cty.vars)]))/sqrt(sample.size))
(cty.sds.sep = sqrt(tapply(y,county,var)/sample.size))

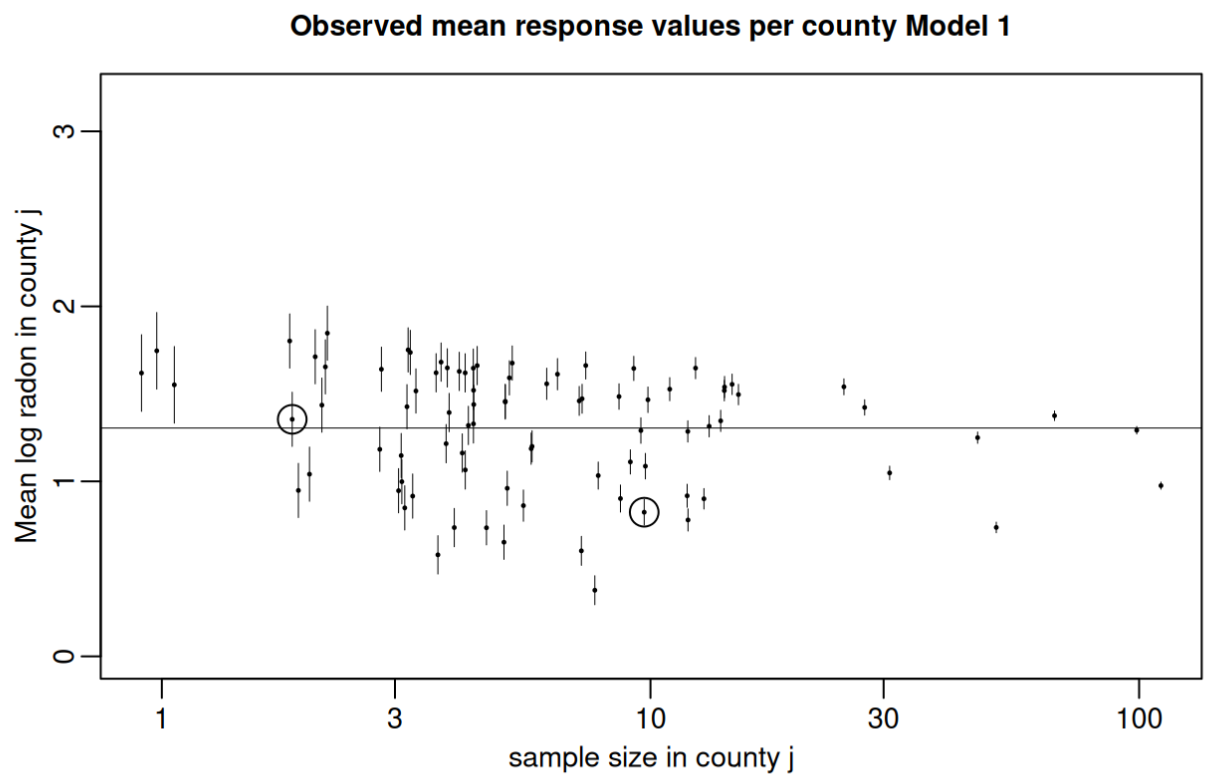
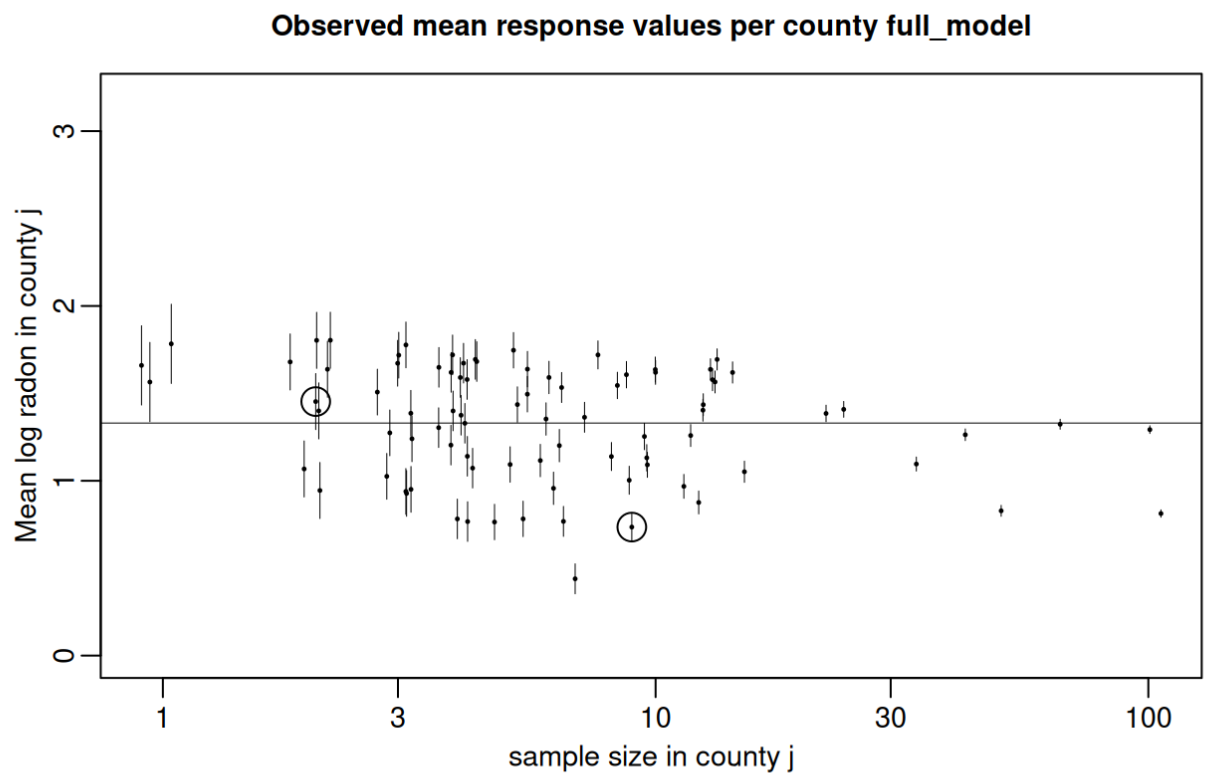
par(mfrow=c(1,1))
plot (sample.size.jittered, cty.mns, cex.lab=.9, cex.axis=1,
      xlab="sample size in county j",
      ylab="Mean log radon in county j",
      pch=20, log="x", cex=.3, mgp=c(1.5,.5,0),
      ylim=c(0,3.2), yaxt="n", xaxt="n")
axis (1, c(1,3,10,30,100), cex.axis=.9, mgp=c(1.5,.5,0))
axis (2, seq(0,3), cex.axis=.9, mgp=c(1.5,.5,0))
for (j in 1:J){
  lines (rep(sample.size.jittered[j],2),
        cty.mns[j] + c(-1,1)*cty.sds[j], lwd=.5)
  #      cty.mns[j] + c(-1,1)*mean(cty.sds[!is.na(cty.sds)]), lwd=.5)
}
abline(h=mean(cty.mns),lwd=.5 )
title("Observed mean response values per county Model 1",cex.main=.9, line=1)

cty.mns[cty.mns==max(cty.mns)]
cty.mns[cty.mns==min(cty.mns)]

```

```
points(sample.size.jittered[36],cty.mns[36],cex=2) #max  
points(sample.size.jittered[37],cty.mns[37],cex=2) #min  
detach(df_pred_before_random)  
})
```





There should be at max two differnt outcomes county

```
df_pred %>%
  group_by(county) %>%
  summarise(n_unique_y = n_distinct(y)) %>% head(10)
```

```
## # A tibble: 10 × 2
##   county n_unique_y
##   <fct>     <int>
## 1 1         2
## 2 2         2
## 3 3         2
## 4 4         2
## 5 5         2
## 6 6         1
## 7 7         2
## 8 8         2
## 9 9         2
## 10 10        2
```

```
df_pred %>%
  group_by(county) %>%
  summarise(
    y_values = list(sort(unique(y))),
    diff_y = diff(sort(unique(y)))
  ) %>% head(10)
```

```
## Warning: Returning more (or less) than 1 row per `summarise()` group was deprecated in
## dplyr 1.1.0.
## i Please use `reframe()` instead.
## i When switching from `summarise()` to `reframe()`, remember that `reframe()`
## always returns an ungrouped data frame and adjust accordingly.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## `summarise()` has grouped output by 'county'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 10 × 3
## # Groups:   county [10]
##   county y_values diff_y
##   <fct> <list>     <dbl>
## 1 1      <dbl [2]> 0.668
## 2 2      <dbl [2]> 0.668
## 3 3      <dbl [2]> 0.668
## 4 4      <dbl [2]> 0.668
## 5 5      <dbl [2]> 0.668
## 6 7      <dbl [2]> 0.668
## 7 8      <dbl [2]> 0.668
## 8 9      <dbl [2]> 0.668
## 9 10     <dbl [2]> 0.668
## 10 14     <dbl [2]> 0.668
```

```
overall_mean <- mean(df_pred$y)

df_pred %>%
  group_by(county) %>%
  summarise(
    county_mean = mean(y),
    diff_from_overall = county_mean - overall_mean
  ) %>%
  arrange(desc(diff_from_overall)) %>%
  head(10)
```

```
## # A tibble: 10 × 3
##   county county_mean diff_from_overall
##   <fct>      <dbl>          <dbl>
## 1 73         1.80           0.579
## 2 65         1.80           0.579
## 3 50         1.78           0.559
## 4 52         1.78           0.552
## 5 31         1.75           0.522
## 6 51         1.72           0.495
## 7 41         1.72           0.495
## 8 6          1.72           0.494
## 9 40         1.69           0.470
## 10 7         1.69           0.469
```

III b) Present a plot of the county level of uranium in the horizontal axis, and the estimated regression intercepts with CI, in the vertical axis. Additionally, one with the estimated regression slopes with CI.

```
model <- lmer(y ~ floor + u.full + (1|county) , df)

library(lme4)
library(ggplot2)

# Random intercepts with conditional variance
re <- ranef(model, condVar=TRUE)
u <- attr(re$county, "postVar") # 1x1xN array of variances

# Random-intercept estimates
intercepts <- re$county[, "(Intercept)"]

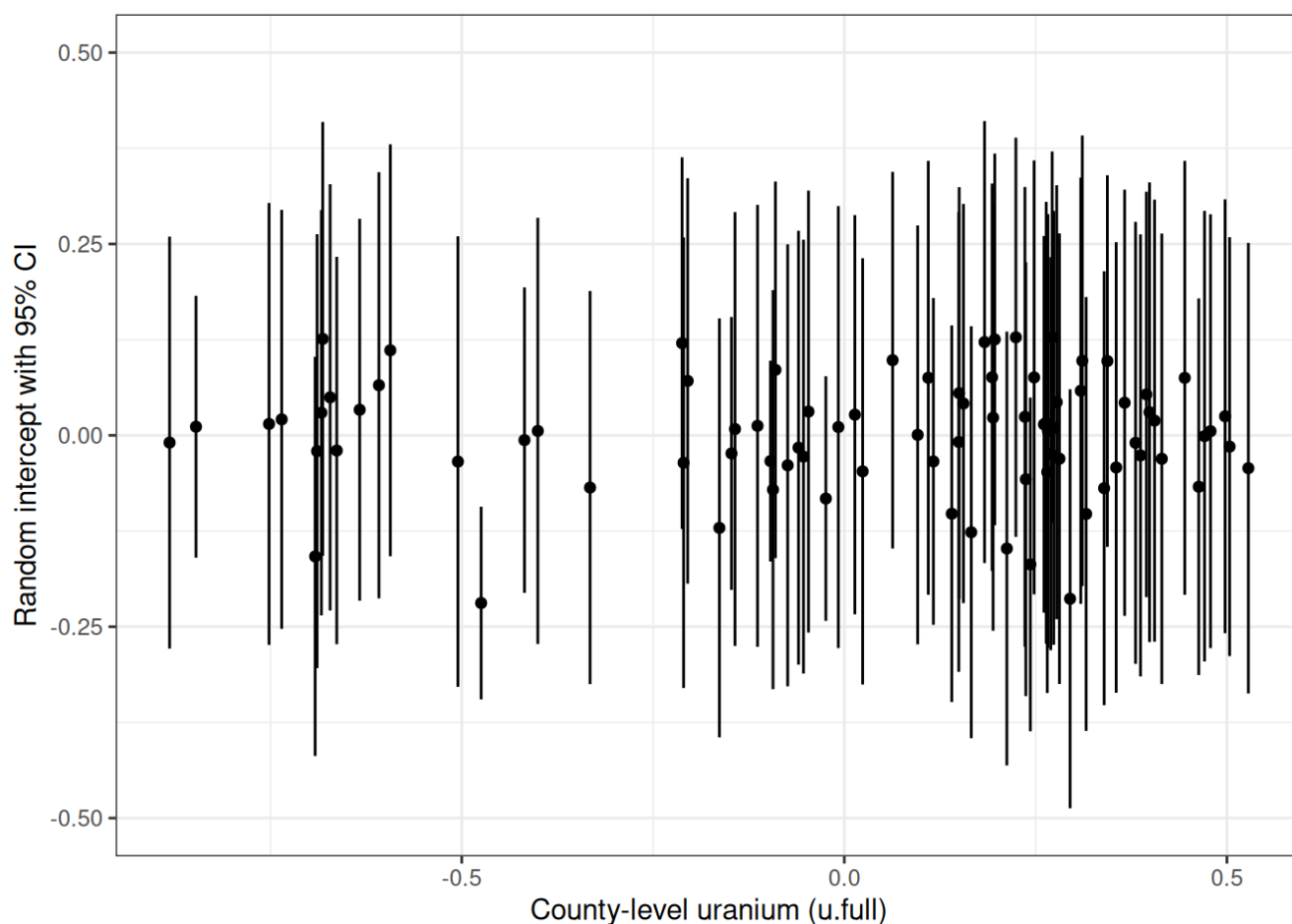
# Std. errors from conditional modes
se <- sqrt(sapply(1:dim(u)[3], function(i) u[,i]))

# Build dataframe
df_plot <- data.frame(
  county = rownames(re$county),
  intercept = intercepts,
  lower = intercepts - 1.96 * se,
  upper = intercepts + 1.96 * se
)

u_by_county <- aggregate(u.full ~ county, data=df, FUN=unique)
names(u_by_county)[2] <- "u_full_county"

df_plot$u_full <- u_by_county$u_full_county[
  match(df_plot$county, u_by_county$county)
]

ggplot(df_plot, aes(x = u_full, y = intercept)) +
  geom_point() +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0) +
  xlab("County-level uranium (u.full)") +
  ylab("Random intercept with 95% CI") +
  ylim(-0.5, 0.5) +
  theme_bw()
```



```
# Posterior variance for each county's random intercept
post_var <- sapply(1:dim(u)[3], function(i) u[, ,i])

# Build a dataframe with county, variance, SE, and intercept
var_by_county <- data.frame(
  county    = rownames(re$county),
  intercept = re$county[, "(Intercept)"],
  var       = post_var,
  se        = sqrt(post_var)
)

# Counties with the *lowest* conditional variance
var_by_county_lowest <- var_by_county %>%
  arrange(var)

# View the top few with lowest variance
head(var_by_county_lowest, 10)
```



```
##      county    intercept      var      se
## 1       70 -0.219046443 0.004123054 0.06421101
## 2       26 -0.033611813 0.004475958 0.06690260
## 3       19 -0.082565464 0.006648864 0.08154057
## 4        2  0.011245769 0.007617373 0.08727756
## 5       80 -0.023622969 0.008274841 0.09096615
## 6       61 -0.006241583 0.010361603 0.10179196
## 7       71 -0.034072808 0.011856612 0.10888807
## 8       54 -0.168591346 0.012366403 0.11120433
## 9        7  0.128109354 0.015333108 0.12382693
## 10      14  0.097050733 0.015333108 0.12382693
```

```
var_by_county_highest <- var_by_county %>%
  arrange(desc(var))

# View the top counties with the largest variance
head(var_by_county_highest, 10)
```

```
##      county    intercept      var      se
## 1       42 -0.008669534 0.02346388 0.1531792
## 2       50  0.030191589 0.02346388 0.1531792
## 3       82  0.024239392 0.02346388 0.1531792
## 4       16 -0.034266810 0.02254429 0.1501475
## 5       23 -0.030629813 0.02254429 0.1501475
## 6       36  0.097433110 0.02254429 0.1501475
## 7       47 -0.035796332 0.02254429 0.1501475
## 8       60 -0.030481513 0.02254429 0.1501475
## 9       65 -0.042872991 0.02254429 0.1501475
## 10      73 -0.001026018 0.02254429 0.1501475
```

III c) Present a plot of the county level of uranium in the horizontal axis, and the estimated regression intercepts with CI, in the vertical axis. Additionally, one with the estimated regression slopes with CI.

```

# 6 in sample
# 6 out of sample

new_data <- data.frame(
  county = c(
    35,35,
    43,43,
    65,65,
    100,100,
    101,101,
    102,102
  ),
  floor = c(
    0,1,
    0,1,
    0,1,
    0,1,
    0,1,
    0,1
  ),
  u.full = c(
    -0.8818289, -0.8818289,
    0.013764829,0.013764829,
    0.52802487,0.52802487,
    -1,-1,
    0,0,
    0.7,0.7
  )
)

new_data$floor <- factor(new_data$floor)
new_data$county <- factor(new_data$county)
new_data$pred_conditional <- predict(model_3, newdata = new_data, re.form = NA)

new_data

```

```

##      county floor      u.full pred_conditional
## 1      35      0 -0.8818289      0.83061001
## 2      35      1 -0.8818289      0.16236525
## 3      43      0  0.01376483      1.47567717
## 4      43      1  0.01376483      0.80743241
## 5      65      0  0.52802487      1.84608202
## 6      65      1  0.52802487      1.17783726
## 7     100      0 -1.00000000      0.74549519
## 8     100      1 -1.00000000      0.07725043
## 9     101      0  0.00000000      1.46576281
## 10     101      1  0.00000000      0.79751805
## 11     102      0  0.70000000      1.96995014
## 12     102      1  0.70000000      1.30170538

```

```
# install.packages("merTools")
library(merTools)
```

```
## Loading required package: arm
```

```
##
## arm (Version 1.14-4, built: 2024-4-1)
```

```
## Working directory is /home/husted42/Documents/github/businessAnalytics/Correlated Data/Exam
```

```
##
## Attaching package: 'arm'
```

```
## The following object is masked from 'package:mvtnorm':
##
##      standardize
```

```
new_data$floor <- factor(new_data$floor)
new_data$county <- factor(new_data$county)
```

```
pred_ci <- predictInterval(
  model,
  newdata = new_data,
  which   = "full",
  level   = 0.95,
  n.sims  = 1000,
  include.resid.var = TRUE
)
```

```
## Warning:      The following levels of county from newdata
## -- 100, 101, 102 -- are not in the model data.
##      Currently, predictions for these values are based only on the
## fixed coefficients and the observation-level error.
```

```
new_data <- cbind(new_data, pred_ci)
```

```

library(lme4)

## Design matrix for the fixed-effects part of the model
## This must match the fixed-effects formula: ~ floor + u.full
X <- model.matrix(~ floor + u.full, data = new_data)

## Fixed-effect estimates and their covariance matrix
beta_hat <- fixef(model)
V_beta <- vcov(model)

## Standard errors of the linear predictor X * beta
pred_se <- sqrt(diag(X %*% V_beta %*% t(X)))

## 95% confidence intervals
new_data$pred_se <- pred_se
new_data$lower_ci <- new_data$pred_conditional - 1.96 * pred_se
new_data$upper_ci <- new_data$pred_conditional + 1.96 * pred_se

library(ggplot2)

ggplot(new_data, aes(x = u.full, y = pred_conditional, colour = county)) +
  geom_point() +
  geom_errorbar(aes(ymin = lower_ci, ymax = upper_ci), width = 0) +
  xlab("u.full") +
  ylab("Predicted y (95% CI, fixed effects only)") +
  theme_bw()

```

